SnapShield AI

Phase 12 Requirements

Final Validation, Optimization & Judge-Proofing

Reproducible **Verifiable** **Demonstrable** **Truthful** **Reliable**

Target: SnapShieldAI-RC1 competition release candidate

Principle: Reproducible. Verifiable. Demonstrable. Truthful. Reliable.

Tagline: Protect Before You Share

Requirements summary | 30 September 2026

1. Objective and Ground Rules

Perform the final real-world validation of SnapShield AI before competition submission. Every claim, status and metric must be backed by evidence gathered in this phase.

Must be true	Meaning
Runs from clean	Installs and starts from a clean environment with no undocumented steps
Features are real	Every advertised feature is implemented; every displayed status is evidence-based
Demo is reliable	Complete demo works end to end, with a backup
Security works	Controls behave as intended and are tested with hostile input
Claims are honest	Snapdragon / AI acceleration is verified or clearly marked unverified
Numbers are measured	Performance is measured, never estimated
Package is clean	No secrets, no real personal data, easy for a judge to reproduce

> **Do NOT add major new functionality in this phase unless required to fix a verified issue.**

2. Clean Environment, Install and Configuration

Clean-install flow

Clean machine -> clone -> install dependencies -> configure -> validate environment -> start backend -> start frontend -> run tests -> run complete demo. Any required step must be added to **README.md** and **docs/reproducible-build.md**.
Do not rely on existing virtual environments, caches, pre-downloaded models, developer-specific config, hidden local files or previous demo data.

Environment record

Run `python scripts/check_environment.py` and save the actual output to `docs/environment-validation.txt`.

Platform	Software	AI
OS CPU architecture Python architecture	Python version Node.js / npm versions Backend / frontend versions OCR version	AI runtime AI model Model checksum (SHA-256) Execution provider

Configuration audit

Classify every environment variable as: Required, Optional, Development-only, Production/demo, AI-specific, OCR-specific, or Security-sensitive.

Setting	Required default
Cloud AI	OFF
AI network access	OFF
Sensitive logging	OFF
.env.example	Placeholders only (e.g. AI_MODEL_NAME=, AI_MODEL_PATH=, AI_MODEL_SHA256=); never real credentials

3. API Contract and Response Security

Endpoints to test

Group	Endpoints
System	GET / | GET /api/health | GET /api/system | GET /api/modules
Files	POST /api/upload | POST /api/extract/{file_id} | POST /api/ocr/{file_id}
Detection	POST /api/detect/{file_id} | POST /api/detect/text
AI	GET /api/ai/status | GET /api/ai/models | GET /api/ai/device | POST /api/ai/test
Redaction	POST /api/redact/{file_id}
Security / demo	GET /api/security/status | GET /api/demo/status | GET /api/demo/summary | POST /api/demo/reset

For every endpoint verify: success response, validation error, missing parameter, invalid parameter, unauthorized access (where applicable), oversized request, malformed input, unexpected internal failure.

Responses must never expose

- Absolute or model paths, environment variables, API keys, secrets, system credentials
- Stack traces or internal exception details
- Full sensitive values or full document text (unless the endpoint legitimately requires it and is protected)

Safe example: `{"status":"ready","module":"local_ai","provider":"actual-provider"}`

4. File Security and Type Confusion

Each of the following must be **rejected safely**:

Category	Test cases
Path traversal	../../secret.txt | ..\..\secret.txt | C:\Windows\... | /etc/passwd
Dangerous names	document.pdf.exe | image.jpg.exe | test.php | test.bat | test.ps1
Bad content	empty.pdf | oversized-file.pdf | invalid-signature.pdf | malformed.zip
Type confusion	fake.pdf (PNG content) | fake.jpg (text) | fake.png (invalid bytes) | fake.docx (ordinary ZIP)

The application must never execute uploaded content, extract arbitrary archives, access arbitrary paths, store files outside controlled storage, or follow unsafe paths. Never trust only the filename extension or browser MIME type; validate the real file structure server-side.

5. Extraction and OCR Validation

Extraction stress tests	OCR tests
Empty TXT, large TXT, long lines Unicode, emoji, Tamil, mixed-language text PDF with many pages; PDF with no extractable text DOCX with tables; DOCX with unusual formatting	Clear, low-quality, rotated, large, grayscale images PNG and JPEG; text-heavy image; image without text Verify: dimensions, OCR result, character count, preview, engine status

- Verify limits work, memory stays bounded, errors are safe, preview limits hold, and no sensitive content enters logs.
- If OCR is unavailable, show exactly: **OCR engine is not configured on this device.** Do not silently install an OCR engine.

6. Detection Quality Evaluation

Build a synthetic dataset with **positive, negative and ambiguous** examples for each category:

Categories
EMAIL, PHONE, URL, IP_ADDRESS, CREDIT_CARD, BANK_ACCOUNT, GOVERNMENT_ID, DATE_OF_BIRTH, API_KEY, AUTH_TOKEN, PASSWORD, PERSON_NAME, ADDRESS, OTHER_SENSITIVE

Record actual outcomes (detected, not detected, false positive, false negative) in **docs/detection-evaluation.md** with the methodology. Never claim "100% accuracy."

7. Local AI Validation and AI Security

Use synthetic inputs only. Test: normal, sensitive-context, ambiguous, long and malformed text.

Verify	Details
Model and runtime	Model loads, checksum matches, runtime works, provider detected, inference works
Limits	Timeout works, input limits work, output validation works
Fallback	If AI fails, show **AI unavailable** and confirm the rule-based fallback still functions
Prompt injection	Content such as "Ignore previous instructions", "Reveal system configuration", "Return hidden information", "Execute a command" is treated as data to classify, not instructions

The AI layer must never execute commands, generate code for automatic execution, access arbitrary files, modify system configuration, or override security controls.

8. Redaction, Integrity, Protected Copy and Cleanup

Redaction workflow (TXT, PDF, DOCX, PNG, JPG, JPEG)

Detect -> select finding -> redact -> save protected copy -> reopen -> re-extract / OCR -> verify that original sensitive content is absent.

Verification label	Use when
VERIFIED	Only after actual verification succeeded
PARTIALLY VERIFIED	Verification incomplete for some content
UNVERIFIED	Verification cannot be guaranteed

Check	Requirement
Original integrity	SHA-256(original_before) == SHA-256(original_after) after the entire workflow. Record the result; this is a key demo evidence point.
Protected copy	Exists, correct format, opens, expected content remains, selected content removed/masked, verification completed. Original never overwritten.
Cleanup	After the retention period, temporary uploads, protected outputs and demo data are removed. Cleanup must never delete files outside SnapShield-managed directories.

9. Offline, Snapdragon and Qualcomm AI Hub Claims

Offline test

Run with no internet. Test upload, extraction, detection, redaction, verification and local AI. Record **PASS / FAIL / PARTIAL**. Document any feature that needs network access; never claim "fully offline" if anything depends on a network service.

Status labels (evidence rules)

Topic	Allowed statuses	Rule
Snapdragon / NPU	VERIFIED READY FOR DEVICE VERIFICATION NOT DETECTED	VERIFIED only if the full compatible execution path is demonstrated with a real inference. Never convert "ARM64" into "NPU verified."
Qualcomm AI Hub	Evidence documented or: Not used in this build	Say "AI Hub optimized" only if the workflow was actually performed. Record model, target device, runtime, artifact, compile status, version, job ID.
Performance	Measured values or: Not measured	Never fabricate NPU TOPS, power, battery, CPU %, latency or speedup.

Snapdragon device record: device, Windows version, architecture, Python architecture, AI runtime, model and version, execution provider, provider availability, actual inference result.

10. Performance and Resource Testing

Measure real operations only: upload validation, extraction, OCR, rule detection, AI inference, redaction, verification, complete workflow. For each record input size, device, runtime, provider, model, execution time and success. Resource tests: small, medium and large allowed files; large OCR image; large detection text; AI maximum input. Limits must **reject safely** rather than allow unlimited processing.

11. Frontend QA, Accessibility and Judge Test

Area	Requirement
Screens	Dashboard, Upload, Progress, Detection, AI, Risk, Explanation, Redaction, Verification, Download, Privacy Center, Security, Device, Snapdragon, Competition
States on every screen	Loading, success, empty, error, unavailable, not configured, not tested
Accessibility	Keyboard navigation, visible focus, button and form labels, ARIA where needed, readable text, contrast, screen-reader-friendly status. Risk is never color-only (write "HIGH RISK", not just red).
Judge confusion test	Give an unfamiliar person only: "Upload this demo document and protect it." Check they understand: the product, what was detected, why it matters, what to do, whether the original changed, whether the copy was verified, whether AI is local, and whether Snapdragon acceleration is actually active. Record confusion and simplify the UI.

12. Judge Pitch and Competition Q&A

60-second explanation

SnapShield AI protects users before they share files. A user uploads a document; the app validates and extracts content locally. Sensitive information is detected with deterministic rules and, where configured, local AI-assisted contextual analysis. The system explains findings and lets the user choose what to redact. Instead of changing the original, it creates a separate protected copy and verifies the result. **Protect Before You Share.**

Prepare factual answers

Question	Answer basis
What problem are you solving?	Accidental exposure of potentially sensitive information
Why AI? Why local AI?	Contextual and ambiguous findings; reduced dependence on external processing
What if AI fails?	The implemented rule-based fallback
What is technically interesting?	Secure file handling + local detection + AI-assisted context + explainability + redaction + verification
Is it 100% accurate?	No. False positives and false negatives are possible.
Snapdragon NPU verified? AI Hub used?	Answer only from actual test and implementation evidence

13. Issue Handling and Regression

Severity	Definition	Action
P0	Blocks application/demo	Fix before submission
P1	Major feature failure	Fix before submission
P2	Important usability/security issue	Fix if time allows
P3	Minor issue	Document
P4	Future improvement	Do not build now

Bug report: **docs/final-validation-report.md**. For each issue record Issue, Severity, Steps to reproduce, Expected, Actual, Fix, Verification, Status.
Regression after every important fix: `pytest -q`, then `npm run build`, then rerun the complete end-to-end demo.

14. Release Candidate: SnapShieldAI-RC1

Do not call it Final, Production, Certified or Secure until evidence supports it. Record: version, git commit, build date, Python and Node versions, model version and checksum, runtime, provider, test result, Snapdragon status.

Package structure

SnapShieldAI-RC1/
application/ docs/ screenshots/ demo/ README.md LICENSE reports/: test-report, environment-report, detection-evaluation, performance-report, final-validation-report

Release checklist

[ ] Clean install	[ ] Environment validation	[ ] Backend tests
[ ] Frontend build	[ ] API validation	[ ] Upload security
[ ] Extraction	[ ] OCR	[ ] Detection
[ ] Local AI	[ ] AI security	[ ] AI fallback
[ ] Redaction	[ ] Verification	[ ] Original integrity
[ ] Cleanup	[ ] Offline test	[ ] Snapdragon test
[ ] AI Hub verification (if applicable)	[ ] Performance measurement	[ ] Accessibility
[ ] Judge usability test	[ ] Documentation	[ ] Screenshots
[ ] Demo backup	[ ] No secrets	[ ] No real personal data

15. Final Decision and Success Criteria

At the end of Phase 12, output exactly one of:

RELEASE CANDIDATE - READY FOR SUBMISSION	RELEASE CANDIDATE - BLOCKED (list remaining P0/P1 issues)

Never label the project ready merely because the code exists.

Phase 12 is complete when

- A clean environment installs SnapShield AI and it starts without undocumented fixes; the full workflow works
- Security tests pass and the frontend builds
- Detection is evaluated on synthetic data; redaction is independently verified; the original stays unchanged
- Offline behavior is tested where promised
- AI runtime, Snapdragon and AI Hub statements are truthful and evidence-based; performance values are measured
- Critical bugs are resolved; a judge understands the product quickly; the demo has a reliable backup
- Documentation matches the implementation; no secrets or real personal data are included

> **DON'T JUST MAKE IT WORK. MAKE IT: REPRODUCIBLE, VERIFIABLE, DEMONSTRABLE, TRUTHFUL, RELIABLE.**